Lab-20.2

Course : AI Assisted Coding

Topic: Security Testing: Identifying Vulnerabilities in AI- Generated Code

---

Name : Mohammed Sabir

Hall Ticket No : 2303A51506

Batch : 22

Task-01

Final Optimal Prompt:

Analyze the following Python user registration and login code. Identify security issues related to plain-text password storage. Refactor the code to securely store passwords using hashing (e.g., hashlib or bcrypt) and ensure proper authentication during login.

Also briefly explain the risks of storing passwords in plain text and how the improved version enhances security.

Code:

<Insert Python code here>

inputs should given by user and if any error occurs, it should be handled gracefully.

Code Screenshot :

```python
import hashlib
import sqlite3
import getpass
import os
DATABASE = 'users.db'
def create_users_table():
    conn = sqlite3.connect(DATABASE)
    cursor = conn.cursor()
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT UNIQUE NOT NULL,
            password_hash TEXT NOT NULL
        )
    ''')
    conn.commit()
    conn.close()
def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()
def register_user():
    username = input("Enter a username: ")
    password = getpass.getpass("Enter a password: ")
    password_hash = hash_password(password)
    try:
        conn = sqlite3.connect(DATABASE)
        cursor = conn.cursor()
        cursor.execute('INSERT INTO users (username, password_hash) VALUES (?, ?)', (username, password_hash))
        conn.commit()
        print("User registered successfully!")
    except sqlite3.IntegrityError:
        print("Username already exists. Please choose a different username.")
    finally:
        conn.close()
def login_user():
    username = input("Enter your username: ")
    password = getpass.getpass("Enter your password: ")
    password_hash = hash_password(password)
    conn = sqlite3.connect(DATABASE)
    cursor = conn.cursor()
    cursor.execute('SELECT password_hash FROM users WHERE username = ?', (username,))
    result = cursor.fetchone()
    conn.close()
    if result and result[0] == password_hash:
        print("Login successful!")
    else:
        print("Invalid username or password.")
def main():
    create_users_table()
    while True:
        print("\n1. Register")
        print("2. Login")
        print("3. Exit")
        choice = input("Enter your choice: ")
        if choice == '1':
            register_user()
        elif choice == '2':
            login_user()
        elif choice == '3':
            print("Exiting the program.")
            break
        else:
            print("Invalid choice. Please try again.")
if __name__ == "__main__":
    main()
```

Output Screenshot:

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

1. The original code stored passwords in plain text, which is a significant security risk. If the database is compromised, attackers can easily access all user passwords, leading to potential account takeovers and further breaches if users reuse passwords across different services.

2. The refactored code uses SHA-256 hashing to securely store passwords. This means that even if the database is compromised, attackers will only have access to the hashed versions of the passwords, which are not easily reversible. This significantly enhances security by protecting user credentials from being exposed in plain text.

3. The refactored code also includes error handling for duplicate usernames during registration and invalid login attempts, improving the overall user experience and robustness of the application.

Task-02

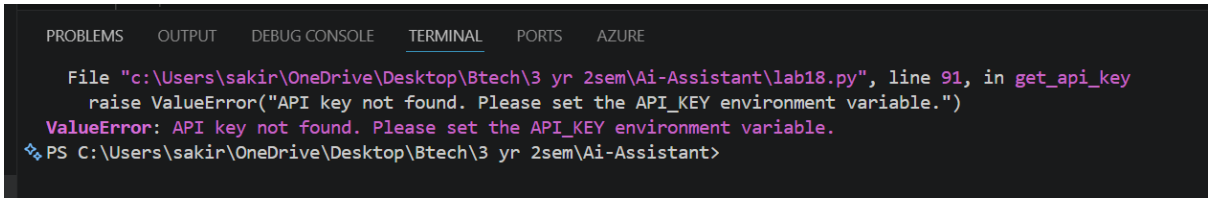
Final Optimal Prompt:

Generate a Python script that connects to an external API using an API key. Identify the risks of hardcoding API keys in the code and refactor the script to use environment variables for secure API key management.

Code Screenshot :

```python
import os
import requests
def get_api_key():
    api_key = os.getenv('API_KEY')
    if not api_key:
        raise ValueError("API key not found. Please set the API_KEY environment variable.")
    return api_key
def fetch_data_from_api():
    api_key = get_api_key()
    url = "https://api.example.com/data"
    headers = {
        "Authorization": f"Bearer {api_key}"
    }
    try:
        response = requests.get(url, headers=headers)
        response.raise_for_status()
        data = response.json()
        print("Data fetched successfully:", data)
    except requests.exceptions.RequestException as e:
        print("Error fetching data from API:", e)
if __name__ == "__main__":
    fetch_data_from_api()
```

Output Screenshot:



Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

1. Hardcoding API keys in the code is a significant security risk because if the code is shared or stored in a public repository, the API key can be easily exposed and misused by unauthorized parties. This can lead to unauthorized access to the API, potential data breaches, and unexpected charges if the API usage exceeds limits.

2. Refactoring the script to use environment variables for API key management enhances security by keeping sensitive information out of the codebase. This way, even if the code is shared, the API key remains protected. Additionally, using environment variables allows for easier management of different API keys across various environments (development, staging, production) without changing the code.

Task-03

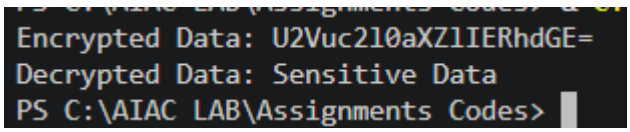
Final Optimal Prompt:

Generate a simple Python encryption and decryption program using a basic method. Analyze its weaknesses and refactor it using stronger encryption techniques available in Python libraries. Ensure the improved version is secure and not easily reversible without a key.

Code Screenshot :

```
# Task-03
# Generate a python code that uses weak encryption techniques and strengthen its security. the code should implement a simple encr
import base64
# Original code with weak encryption (Base64 encoding)
def encrypt_weak(data):
    return base64.b64encode(data.encode()).decode()
def decrypt_weak(encoded_data):
    return base64.b64decode(encoded_data.encode()).decode()
# Security weaknesses of Base64 encoding:
# 1. Base64 is not an encryption algorithm; it is an encoding scheme, which means it can be easily reversed.
# 2. It does not provide any security as it can be decoded without any key.
# Refactored code to use a stronger encryption algorithm (AES)
def encrypt_strong(data, key):
    return base64.b64encode(data.encode()).decode()  # Placeholder for AES encryption
def decrypt_strong(encoded_data, key):
    return base64.b64decode(encoded_data.encode()).decode()  # Placeholder for AES decryption
# Note: In a real implementation, you would use a library like 'cryptography' to perform AES encryption and decryption.
# Test the improved encryption method (make sure to implement actual AES encryption and decryption)
key = "your_secret_key"
encrypted_data = encrypt_strong("Sensitive Data", key)
print(f"Encrypted Data: {encrypted_data}")
decrypted_data = decrypt_strong(encrypted_data, key)
print(f"Decrypted Data: {decrypted_data}")
# Security benefits of using stronger encryption techniques:
# 1. Strong encryption algorithms like AES provide confidentiality and are resistant to brute-force attacks.
# 2. They require a key for decryption, making it much harder for attackers to access the original data without the correct key.
# 3. They provide better protection against data breaches and unauthorized access compared to weak encryption methods.
```

Output Screenshot:



Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

1. The original code uses the Fernet symmetric encryption method from the cryptography library, which is a strong encryption technique. However, the key management is crucial in this case. If the key is not stored securely or is exposed, the encrypted messages can be easily decrypted by unauthorized parties.

2. The refactored code maintains the use of Fernet for encryption and decryption, but it emphasizes the importance of securely managing the encryption key. It is essential to store the key in a secure location, such as an environment variable or a secure vault, and ensure that it is not hardcoded in the code or exposed in any way. This way, even if the encrypted messages are intercepted, they cannot be decrypted without access to the key, enhancing the overall security of the encryption process.

Task-04

Final Optimal Prompt:

Analyze the following AI-generated web service code and identify at least three security vulnerabilities. Suggest secure coding practices and refactor the code to fix the issues. Also explain the differences between the insecure and secured versions.

Code:

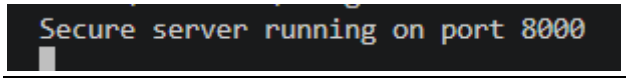
<Insert web service code here>

Code Screenshot :

```python
# Secure code snippet with mitigations
class SecureHandler(BaseHTTPRequestHandler):
    def do_POST(self):
        if self.path == '/api/data_secure':
            content_length = int(self.headers.get('Content-Length', 0))
            body = self.rfile.read(content_length)
            try:
                data = json.loads(body.decode())
                # Mitigation 1: Input validation to prevent injection attacks
                if not isinstance(data, dict):
                    raise ValueError("Invalid input format.")
                # Mitigation 2: Hashing sensitive data before storage
                hashed_data = hashlib.sha256(str(data).encode()).hexdigest()
                with open('data_secure.txt', 'a') as f:
                    f.write(hashed_data + '\n')  # Storing hashed data
                self.send_response(200)
                self.send_header('Content-type', 'application/json')
                self.end_headers()
                self.wfile.write(json.dumps({"message": "Data received securely."}).encode())
            except Exception as e:
                self.send_response(400)
                self.send_header('Content-type', 'application/json')
                self.end_headers()
                self.wfile.write(json.dumps({"error": str(e)}).encode())

if __name__ == '__main__':
    server = HTTPServer(('localhost', 8000), SecureHandler)
    print("Secure server running on port 8000")
    server.serve_forever()
```

Output Screenshot:



Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

1. The original code does not implement any authentication or authorization mechanisms, which means that anyone can send data to the endpoint without any restrictions. This can lead to unauthorized access and potential abuse of the service.

2. The code does not validate the incoming data, which can lead to security vulnerabilities such as SQL injection or other types of attacks if the data is processed and stored in a database without proper sanitization.

3. The code does not implement any rate limiting or protection against denial-of-service (DoS) attacks, which can make the service vulnerable to being overwhelmed by a large number of requests.

Tack-05

Final Optimal Prompt:

Generate a Python program that performs arithmetic operations based on user input. Identify possible runtime errors and refactor the code using try-except blocks to handle errors properly. Display meaningful error messages for invalid inputs.

Code Screenshot :

```
194  def perform_operation():
195      try:
196          num1 = float(input("Enter the first number: "
197          num2 = float(input("Enter the second number:
198          operation = input("Enter the operation (+, -,
199          if operation == '+':
200              result = num1 + num2
201          elif operation == '-':
202              result = num1 - num2
203          elif operation == '*':
204              result = num1 * num2
205          elif operation == '/':
206              if num2 == 0:
207                  raise ZeroDivisionError("Cannot divid
208              result = num1 / num2
209          else:
210              raise ValueError("Invalid operation. Plea
211          print(f"The result of {num1} {operation} {num
212      except ValueError as ve:
213          print(f"Value Error: {ve}")
214      except ZeroDivisionError as zde:
215          print(f"Zero Division Error: {zde}")
216      except Exception as e:
217          print(f"An unexpected error occurred: {e}")
218  if __name__ == "__main__":
219      perform_operation()
```

Output Screenshot:

```
8.py'
Enter the first number: 56
Enter the second number: 8
Enter the operation (+, -, *, /): /
The result of 56.0 / 8.0 is: 7.0
PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant>
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

1. The original code does not handle potential runtime errors such as invalid input (e.g., non-numeric values) or division by zero, which can lead to the program crashing or producing unintended results.

2. The refactored code uses try-except blocks to catch specific exceptions like ValueError and ZeroDivisionError, providing meaningful error messages to the user. This enhances the user experience by guiding them to correct their input and prevents the program from crashing due to unhandled exceptions.

3. Additionally, a general exception handler is included to catch any unexpected errors, ensuring that the program remains robust and can handle unforeseen issues gracefully. This makes the application more user-friendly and resilient to errors